

Agentic AI & Multiagent Systems



Semester Course of 03 Credits

B.Tech. 6th Sem / M.Tech. 2nd Sem

Prof Giovanni Merlino
Associate Professor (Computer Engineering)
University Of Messina, Italy

Prof Dr. Om Prakash Vyas
Vice Chancellor & Director
IIIT Naya Raipur, India

Multi-Agent Coordination (MAC)

Reference: Agentic AI: Theories and Practices (Huang & Huang)

Subtitle: From Individual Intelligence to Collective Synergy

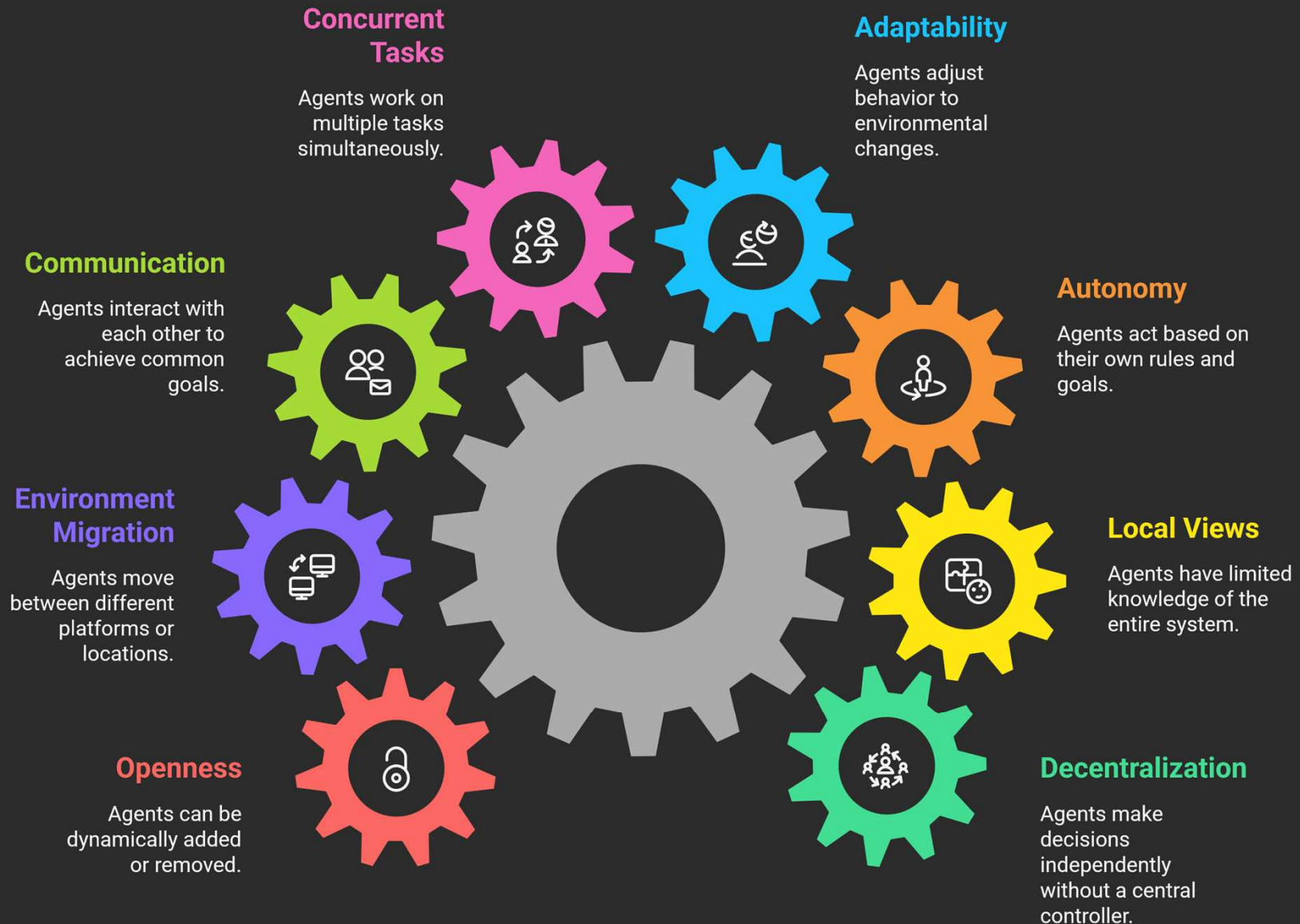
CONTENTS

- Recap of MAS
- Communication of Agent - Fundamentals
- Agent Communication Languages
- Message Transport & Routing
- Semantic Frameworks
- Conflict Resolution in MAS
- Types of Conflicts and Conflict Detection Mechanisms
- Designing Multi-Agent Environments
- Agent Roles & Specialization
- Evolution, Deployment & Retirement
- System Health Monitoring

Multi-Agent Systems

- Multi-Agent Systems (MAS) offer a compelling paradigm shift in how we approach complex problem-solving with AI. By distributing tasks across multiple specialized agents, MAS addresses limitations inherent in monolithic, single-agent architectures.

Multi-Agent System Characteristics



Fundamentals of Agent Communication

Foundations of Agent Communication



Communication Protocols

Predefined rules governing information exchange between agents.



Message Types

Various message formats used by agents to convey intentions.



Semantic Interoperability

Shared understanding of message meaning through ontologies.



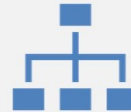
Communication Strategies

Different approaches agents use to communicate effectively.

Designing Scalable MAS Architectures



Modularity: A modular architecture allows for the independent development and deployment of agents, making it easier to scale the system. Each module can represent a specific functionality or agent type, promoting reusability.



Decentralization: Decentralized architectures reduce the risk of bottlenecks and single points of failure. By distributing decision-making and control among agents, the system can better handle increased loads and dynamic changes.



Dynamic Agent Management: The ability to dynamically add or remove agents from the system is essential for scalability. This can be achieved through techniques such as agent migration, where agents can move between different environments or platforms as needed.



Load Balancing: Implementing load balancing mechanisms ensures that no single agent or group of agents becomes overwhelmed with tasks. This can involve distributing tasks evenly across agents or reallocating resources based on current demand.

Agent Communication Languages

Agent Communication Languages provide the semantic framework and syntactic structure for meaningful interaction between agents. ACLs go beyond simple data exchange protocols to incorporate sophisticated mechanisms for expressing intent, knowledge, and queries.

FIPA-ACL is standardized by the Foundation for Intelligent Physical Agents, which ensures interoperability among agents developed by different organizations.

The Foundation for Intelligent Physical Agents (FIPA) Agent Communication Language represents the industry standard for agent communication (Wikipedia, 2024). FIPA-ACL defines a comprehensive set of performatives that enable rich agent interactions. These performatives include inform for sharing information, request for action solicitation, propose for negotiation initiation, agree/refuse for response to proposals, and query-if/query-ref for information gathering.

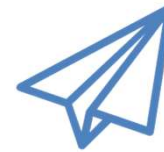
Agent Communication Languages (ACLs)



FIPA-ACL
(performative-based
messaging)



KQML (knowledge
query & manipulation)



Message fields: sender,
receiver, performative,
content



Conversation tracking
via IDs

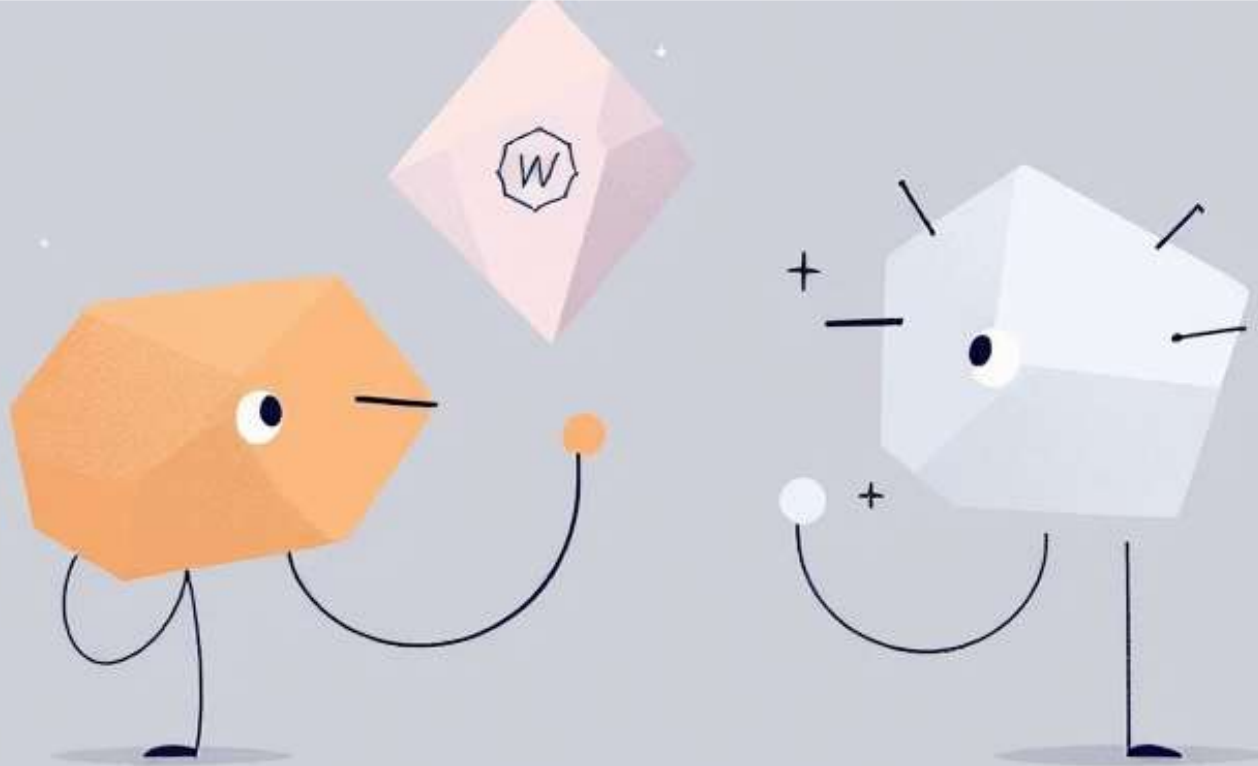
FIPA-ACL

FIPA-ACL is a performative-based messaging framework that allows agents to communicate using a set of predefined performatives. Performatives are communicative acts that convey the intention behind a message, such as requesting information, providing information, or expressing agreement.

Conversation Tracking: FIPA-ACL supports conversation tracking through the use of unique identifiers (IDs). Each message can include a conversation ID that allows agents to keep track of ongoing dialogues and maintain context. This is crucial for managing complex interactions and ensuring that agents respond appropriately to previous messages.

Key Features of FIPA-ACL

- **Performative-Based Messaging:** FIPA-ACL defines a set of performatives that agents can use to express their intentions. Common performatives include:
 - **Inform:** To convey information.
 - **Request:** To ask for information or action.
 - **Agree:** To indicate acceptance of a request.
 - **Reject:** To indicate refusal of a request.
- **Message Structure:** A typical FIPA-ACL message consists of several fields:
 - **Sender:** The agent that is sending the message.
 - **Receiver:** The agent that is intended to receive the message.
 - **Performative:** The type of communicative act being performed.
 - **Content:** The actual information or data being conveyed



KQML

KQML is another widely used Agent Communication Language that focuses on knowledge sharing and manipulation. It is designed to facilitate the exchange of knowledge among agents, making it particularly suitable for applications in distributed artificial intelligence and knowledge-based systems.

Key Features of KQML

1. Knowledge Query and Manipulation: KQML allows agents to perform various operations related to knowledge, such as querying for information, updating knowledge bases, and sharing facts. This makes it a powerful tool for agents that need to collaborate and share insights.

2. Message Structure: Similar to FIPA-ACL, KQML messages consist of several fields:

- **Sender:** The agent that sends the message.
- **Receiver:** The agent that is the target of the message.
- **Performative:** The communicative act being performed, which can include actions like tell, ask, or subscribe.
- **Content:** The knowledge or information being shared.

3. Flexibility and Extensibility: KQML is designed to be flexible and extensible, allowing developers to define new performatives and message structures as needed. This adaptability makes KQML suitable for a wide range of applications and domains.

Message Fields in ACLs

Both FIPA-ACL and KQML utilize similar message fields that are crucial for effective communication between agents. Understanding these fields is essential for developers working with ACLs.

Sender

- The sender field identifies the agent that is initiating the communication. This is important for establishing the origin of the message and for the recipient to know who to respond to.

Receiver

- The receiver field specifies the intended recipient of the message. This allows agents to direct their communications to specific targets, ensuring that messages reach the appropriate audience.

Performative

- The performative field indicates the type of action being communicated. This is a key component of ACLs, as it defines the intention behind the message and guides the recipient's response.

Content

- The content field contains the actual information or data being conveyed in the message. This can include facts, requests, or any other relevant information that the sender wishes to share.

Conversation Tracking Via IDs

Conversation tracking is a vital aspect of agent communication, particularly in complex multi-agent systems. By using unique conversation IDs, agents can maintain context and manage ongoing dialogues effectively.

Importance of Conversation IDs

- **Context Maintenance:** Conversation IDs help agents keep track of the context of interactions. This is especially important in scenarios where multiple conversations may be happening simultaneously.
- **Response Appropriateness:** By referencing conversation IDs, agents can ensure that their responses are relevant to the specific dialogue they are engaged in. This reduces confusion and enhances the clarity of communication.
- **Error Handling:** In cases where messages are lost or misinterpreted, conversation IDs can assist agents in identifying and resolving issues, allowing for more robust communication protocols.

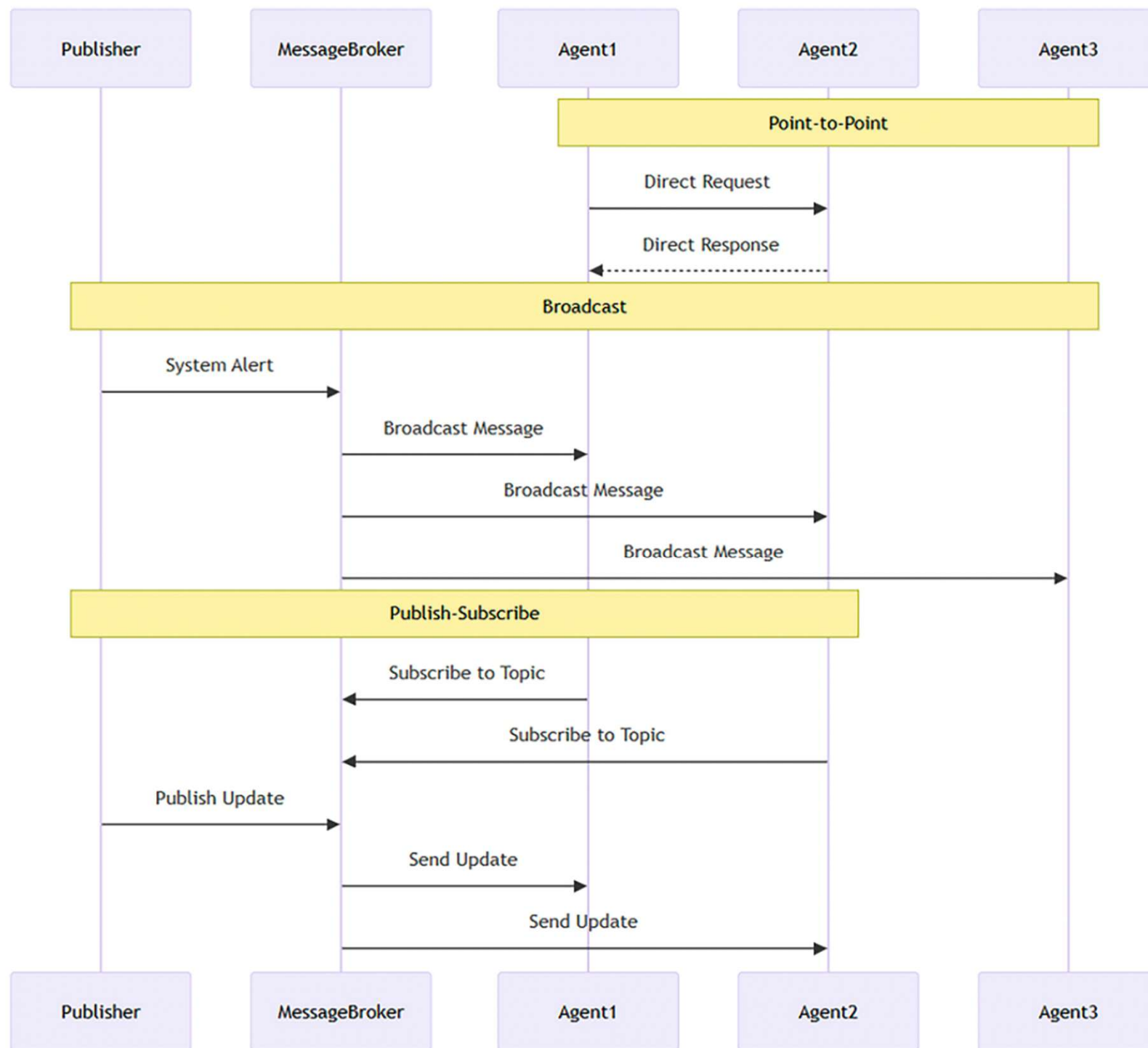


Fig. 3.1 Multi-agent communication patterns

Message Transport & Routing



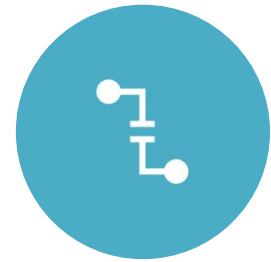
DIRECT POINT-TO-POINT MESSAGING



PUBLISH/SUBSCRIBE MODELS



BLACKBOARD/SHARED WORKSPACE MODEL



CONSIDER LATENCY,
RELIABILITY,
SCALABILITY

Direct Point-to-Point Messaging

Direct point-to-point messaging is one of the simplest forms of communication between agents. In this model, a message is sent directly from one agent to another, establishing a dedicated communication channel. This method is straightforward and allows for immediate feedback, making it suitable for scenarios where quick responses are necessary.

Advantages

- **Simplicity:** The model is easy to implement and understand.
- **Low Latency:** Direct communication often results in lower latency since messages do not need to traverse multiple intermediaries.
- **Reliability:** The direct nature of the communication can enhance reliability, as messages are sent and received without additional layers of complexity.

Disadvantages

- **Scalability Issues:** As the number of agents increases, managing direct connections can become cumbersome and lead to scalability challenges.
- **Tight Coupling:** Agents become tightly coupled, which can hinder flexibility and adaptability in dynamic environments.

Publish/Subscribe Model

The publish/subscribe model is a more flexible communication paradigm that decouples message producers (publishers) from consumers (subscribers). In this model, agents can publish messages to a topic, and any agent interested in that topic can subscribe to receive updates. This approach is particularly useful in scenarios where multiple agents need to be informed about certain events or changes.

Advantages

- **Decoupling:** Publishers and subscribers are independent, allowing for greater flexibility and easier integration of new agents.
- **Scalability:** The model scales well as the number of agents increases, as new subscribers can join without affecting existing connections.
- **Broadcasting:** Messages can be broadcasted to multiple subscribers simultaneously, enhancing efficiency in information dissemination.

Disadvantages

- **Latency:** Depending on the implementation, there may be increased latency due to the need for message routing through a broker or intermediary.
- **Reliability Concerns:** Ensuring message delivery can be more complex, as messages may be lost if subscribers are not available at the time of publication.

Blackboard/Shared Workspace Model

The blackboard/shared workspace model is a collaborative communication approach where agents share a common workspace or blackboard. Agents can read from and write to this shared space, allowing for collective problem-solving and information sharing. This model is particularly effective in scenarios requiring collaboration among multiple agents.

Advantages

- **Collaboration:** The model fosters collaboration among agents, enabling them to build on each other's contributions.
- **Flexibility:** Agents can operate independently while still contributing to a shared goal, allowing for dynamic interactions.
- **Rich Information Sharing:** The shared workspace can hold complex data structures, facilitating rich information exchange.

Disadvantages

- **Latency:** Accessing a shared workspace may introduce latency, especially if multiple agents are trying to read or write simultaneously.
- **Contention:** Multiple agents accessing the same resources can lead to contention issues, potentially affecting performance.
- **Complexity:** Managing the shared workspace and ensuring data consistency can add complexity to the system.

Considerations for Latency, Reliability, and Scalability

When designing agent communication systems, it is essential to consider the following factors:

Latency

- **Network Overhead:** The choice of communication model can significantly impact latency. Direct messaging typically offers lower latency, while publish/subscribe and blackboard models may introduce delays due to routing and access times.
- **Message Size:** Larger messages can increase transmission time, affecting overall latency. Optimizing message size is crucial for maintaining responsiveness.

Reliability

- **Message Acknowledgment:** Implementing acknowledgment mechanisms can enhance reliability, ensuring that messages are received and processed correctly.
- **Redundancy:** Introducing redundancy in message delivery can help mitigate the risk of message loss, particularly in publish/subscribe systems.

Scalability

- **Load Balancing:** In publish/subscribe models, load balancing can help distribute the message load evenly across subscribers, improving scalability.
- **Dynamic Discovery:** Implementing dynamic discovery mechanisms allows agents to join and leave the system without disrupting ongoing communication, enhancing scalability.

Evaluating and Benchmarking MAS Systems



Performance Metrics:

Establishing clear performance metrics is crucial for evaluating MAS systems. Common metrics include response time, throughput, resource utilization, and success rates in achieving goals.



Benchmarking Frameworks:

Utilizing standardized benchmarking frameworks allows for consistent evaluation across different MAS implementations. These frameworks can provide a basis for comparison and help identify best practices.



Simulation and Testing:

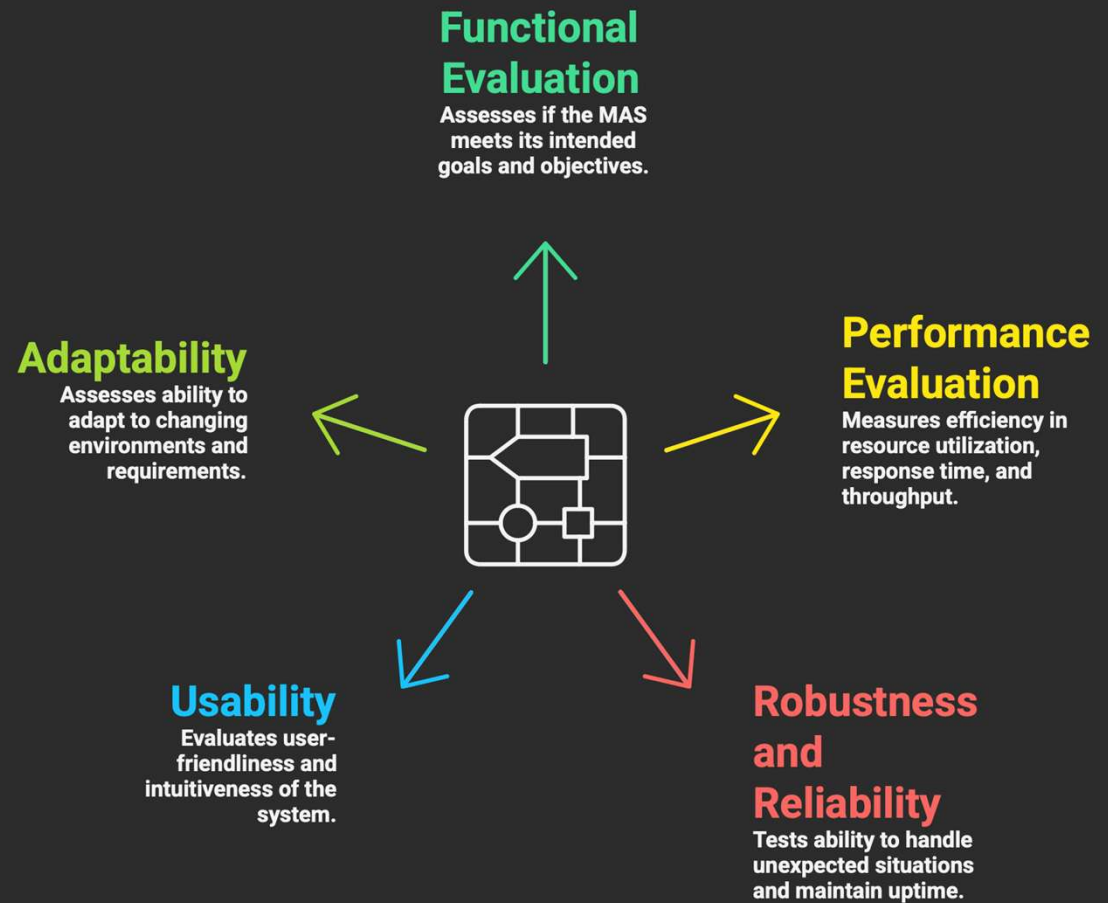
Simulating agent interactions in controlled environments can help identify potential issues and assess system performance under various conditions. Testing can also involve real-world scenarios to validate the system's effectiveness.



User Feedback: Gathering feedback from users and stakeholders can provide valuable insights into the system's usability and effectiveness. This feedback can inform iterative improvements and guide future development efforts.

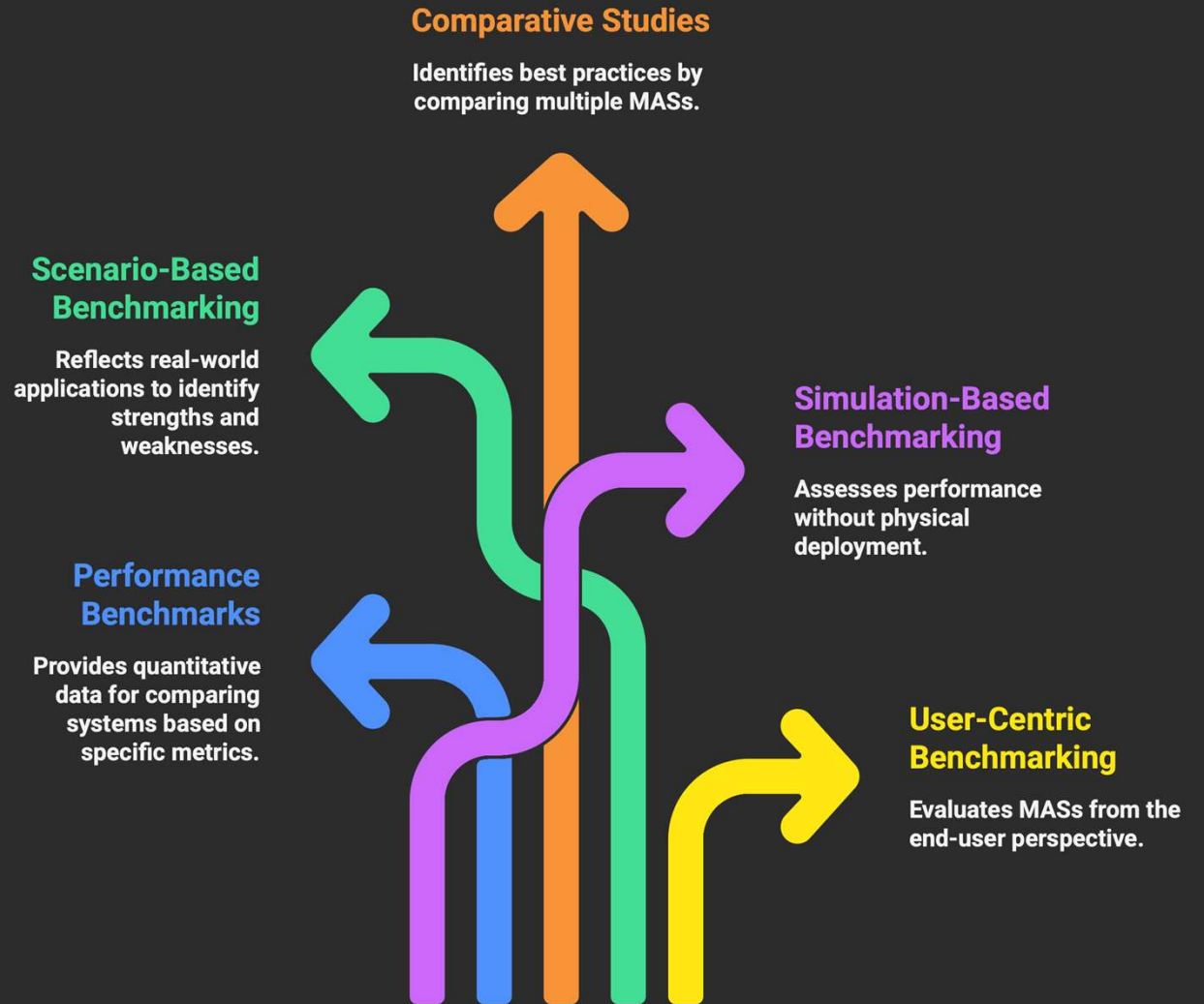
Evaluation and Benchmarking of Multi-Agent Systems (MASs)

MAS Evaluation Dimensions



Benchmarking Methodologies for MAS Evaluation

Benchmarking methodology for MAS evaluation



Performance Analysis techniques

Performance analysis techniques are essential for understanding how well a MAS operates under different conditions. Key techniques include:

- **Profiling:** Profiling involves monitoring the execution of a MAS to identify bottlenecks and areas for optimization. Tools such as profilers can provide insights into resource usage and execution time for different components.
- **Load Testing:** This technique evaluates how a MAS performs under varying loads, simulating different levels of user activity or agent interactions. Load testing helps identify the system's capacity limits and scalability.
- **Stress Testing:** Stress testing pushes a MAS beyond its normal operational limits to assess its behavior under extreme conditions. This technique helps identify potential failure points and ensures the system can recover gracefully.
- **Latency Measurement:** Measuring latency involves assessing the time taken for messages to be transmitted between agents or between agents and users. This metric is crucial for applications requiring real-time responses.
- **Resource Utilization Analysis:** This technique evaluates how efficiently a MAS uses system resources, such as CPU, memory, and network bandwidth. Understanding resource utilization helps optimize system performance and reduce costs.

Standardization and other practices

Standardization and best practices play a crucial role in the evaluation and benchmarking of MAS. Establishing common standards helps ensure consistency and comparability across different systems. Key aspects include:

- **Standard Metrics:** Defining standard metrics for evaluating MAS performance allows for consistent comparisons. Metrics should be relevant, measurable, and applicable across various applications.
- **Best Practice Guidelines:** Developing best practice guidelines for MAS design, implementation, and evaluation can help practitioners avoid common pitfalls and enhance system performance.
- **Interoperability Standards:** Establishing interoperability standards ensures that different MASs can communicate and work together effectively. This is particularly important in applications involving multiple agents from different sources.
- **Documentation and Reporting:** Comprehensive documentation of evaluation and benchmarking processes is essential for transparency and reproducibility. Clear reporting of results allows others to understand and build upon previous work.
- **Community Collaboration:** Encouraging collaboration within the MAS research community can lead to the sharing of knowledge, tools, and methodologies. This collaboration can drive innovation and improve the overall quality of MAS evaluations.

Conflict Detection and Resolution

How to detect and resolve conflicts in a multi-agent system ?





Negotiation Between Agents



Key Aspects of Agent Negotiation

- **Communication Protocols:** Agents must adhere to specific communication protocols to exchange offers, counteroffers, and information effectively. Protocols like Contract Net Protocol (CNP) and Auction-based mechanisms are commonly used.
- **Utility Functions:** Each agent typically has a utility function that quantifies its preferences over possible outcomes. Understanding these utility functions helps agents make informed decisions during negotiations.
- **Bargaining Strategies:** Agents can employ various bargaining strategies, such as cooperative, competitive, or mixed approaches, depending on their objectives and the context of the negotiation.
- **Time Constraints:** Negotiation often occurs under time constraints, which can influence the strategies employed by agents. Time-limited negotiations may lead to quicker agreements but can also result in suboptimal outcomes.



Arbitration via Mediator Agent



Functions of Mediator Agents



Information Gathering: Mediator agents collect information from the conflicting parties to understand their positions and preferences better.



Proposal Generation: Based on the gathered information, the mediator generates potential solutions that consider the interests of all parties involved.



Facilitating Dialogue: The mediator facilitates communication between agents, encouraging them to express their concerns and preferences openly.



Final Decision: In some cases, the mediator may have the authority to make a binding decision if the parties cannot reach an agreement.



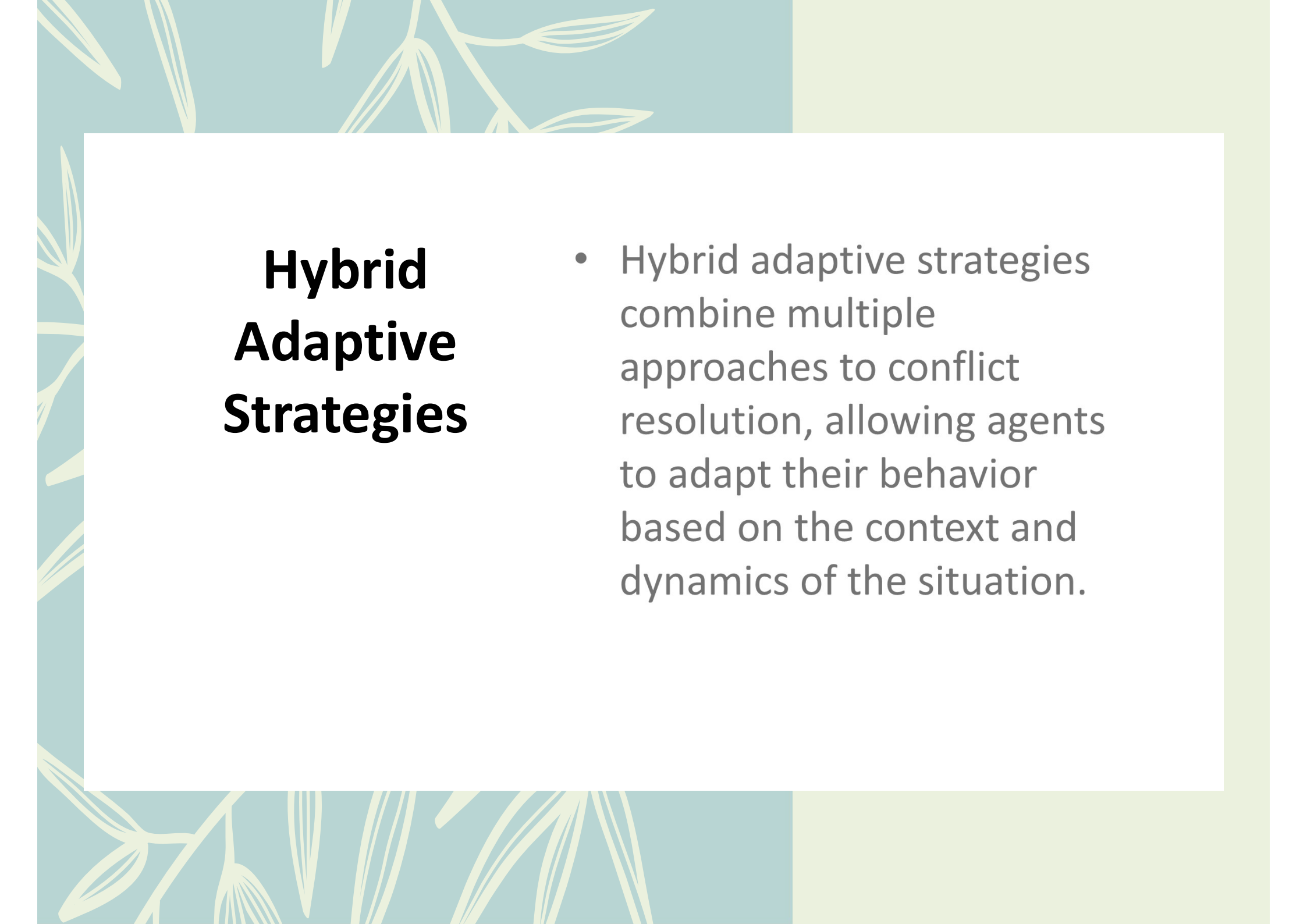
Consensus & Voting Mechanisms

Consensus and voting mechanisms are essential for achieving agreement among multiple agents, particularly in decentralized systems where no single agent has authority over others.

Consensus Mechanisms



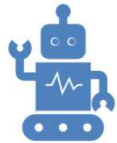
- **Majority Voting:** A simple and widely used method where the option with the most votes is selected. This method is effective in scenarios with clear preferences but may lead to dissatisfaction among minority agents.
- **Weighted Voting:** In this approach, agents have different voting weights based on their importance or stake in the outcome. This method can help ensure that more influential agents have a greater say in the decision-making process.
- **Consensus Algorithms:** Algorithms like Paxos and Raft are designed to achieve consensus in distributed systems, ensuring that all agents agree on a single value or decision despite potential failures or conflicts.



Hybrid Adaptive Strategies

- Hybrid adaptive strategies combine multiple approaches to conflict resolution, allowing agents to adapt their behavior based on the context and dynamics of the situation.

Real World MAS Application



Robotics: In robotic systems, multiple agents can collaborate to perform complex tasks, such as search and rescue operations, warehouse management, and autonomous vehicle navigation.



Smart Grids: MAS can optimize energy distribution in smart grids by enabling agents to communicate and coordinate energy consumption and generation, leading to improved efficiency and reliability.



Healthcare: In healthcare, MAS can facilitate patient monitoring, resource allocation, and treatment planning by allowing agents to share information and collaborate on patient care.



Supply Chain Management: MAS can enhance supply chain operations by enabling agents to coordinate logistics, inventory management, and demand forecasting, leading to reduced costs and improved service levels.



Gaming and Entertainment: In gaming, MAS can create more realistic and dynamic environments by allowing non-player characters (NPCs) to interact and respond to player actions in a coordinated manner.